

Design Patterns

The 8 essential patterns every developer should know — with Java

CREATIONAL how objects get made Singleton • Factory • Builder	STRUCTURAL how objects are arranged Adapter • Decorator • Facade	BEHAVIORAL how objects talk Strategy • Observer
---	--	---

Design patterns are proven solutions to problems that come up again and again in object-oriented code. They share one goal with SOLID: **reduce how far a single change ripples through your code**. Each pattern below has a plain definition, a real-world analogy, the problem it solves, and runnable Java code with its output.

Watch for the recurring move: nearly every pattern's answer is *“program to an interface, so you add a new class instead of editing an old one.”* That's Open/Closed and Dependency Inversion showing up over and over — the patterns are SOLID made concrete.

What's inside

1	Singleton	one shared instance, locked new
2	Factory	one place that decides what to create
3	Builder	build step-by-step, by name, then build()
4	Adapter	translate an incompatible interface
5	Decorator	wrap in layers, each adding behaviour
6	Facade	one simple door over a messy subsystem
7	Strategy	swap the algorithm at runtime
8	Observer	broadcast to many listeners

1

Singleton

Creational — one shared instance, locked new

Definition

A class that is allowed to have **only one object** in the entire program. No matter how many times you ask for it, you always get the *same single copy* back.

The trick: normally you create an object with `new`. Singleton **blocks that door** and gives you one official way in that always hands back the same object.

Real-world analogy

The **principal of a school**. There are many students and teachers, but only **one principal**. If a student, a teacher, and a parent all say “I need the principal”, they all end up talking to the *same person*. Five principals giving five different orders would be chaos.

The problem it solves

Some things should exist only once — app settings, a single connection manager, one logbook the whole app writes to. If every part of the program made its own copy, they'd disagree with each other, like five principals giving five different rules.

Three ingredients: a **private constructor** (nobody outside can say `new`), a **private static variable** (the class holds the one object inside itself), and a **public getInstance()** (the only official way in).

```

class Principal {
    // 2. the class holds the ONE object inside itself
    private static Principal instance;
    private String name;

    // 1. PRIVATE constructor - nobody outside can do `new Principal()`
    private Principal() {
        this.name = "Mr. Sharma";
        System.out.println("The one principal is appointed.");
    }

    // 3. the ONLY official way to get the principal
    public static Principal getInstance() {
        if (instance == null) { // if no principal yet...
            instance = new Principal(); // ...appoint one (happens ONCE)
        }
        return instance; // always the same one
    }

    public void announce(String msg) {
        System.out.println(name + " announces: " + msg);
    }
}

public class Main {
    public static void main(String[] args) {
        // we CANNOT write `new Principal()` - the door is locked
        Principal p1 = Principal.getInstance(); // appoints
        Principal p2 = Principal.getInstance(); // SAME one
        Principal p3 = Principal.getInstance(); // still SAME

        p1.announce("School closes early today.");

        System.out.println("p1 and p2 same? " + (p1 == p2));
        System.out.println("p2 and p3 same? " + (p2 == p3));
    }
}

```

OUTPUT

```

The one principal is appointed.
Mr. Sharma announces: School closes early today.
p1 and p2 same? true
p2 and p3 same? true

```

Read the output carefully

“The one principal is appointed” prints **only once**, even though `getInstance()` was called three times. The first call created it; the next two handed back the *same* object already sitting inside the class. And `p1 == p2` being true proves it — they aren't three principals, they're **three names pointing at one person**.

When to use it

When something must exist **exactly once** and be **shared everywhere**: app settings, a logger, a connection manager.

The SOLID connection

Supports **Single Responsibility** — the class controls its own instantiation. But note it works *against* Dependency Inversion, since callers reach for a concrete global rather than an injected abstraction.

One honest caution: Singleton is sometimes called an **anti-pattern** when overused. Because it's a single global everyone shares, it hides who depends on what and makes testing harder. Use it for genuinely “one-only” things — not as a shortcut to share data everywhere.

Singleton — the thread-safety problem

A **thread** is one worker doing tasks. Modern programs run many workers at once. That speed creates a subtle danger:

X RACE CONDITION

```
if (instance == null) {
    instance = new Principal();
}
// Worker A checks null -> true -> starts creating.
// Worker B checks null at the SAME instant -> also true -> ALSO creates.
// Result: TWO principals. The whole promise is broken.
```

So the “types” of Singleton all exist to answer one question: **how do we guarantee only one object when many threads attack at once?**

Type 1 — Eager Initialization

Create it immediately when the class loads, before any thread can race. Safe, but built even if never used.

```
class Principal {
    // created RIGHT AWAY, once, before anyone asks
    private static final Principal instance = new Principal();
    private Principal() { }
    public static Principal getInstance() { return instance; }
}
```

Type 2 — Synchronized Method

Lazy + safe, but the lock fires on *every* call forever — slow.

```
class Principal {
    private static Principal instance;
    private Principal() { }

    // `synchronized` = only ONE thread inside at a time
    public static synchronized Principal getInstance() {
        if (instance == null) instance = new Principal();
        return instance;
    }
}
```

Type 3 — Double-Checked Locking

Lock only during first creation. Fast forever after.

```

class Principal {
    // `volatile` = all threads see the latest value correctly
    private static volatile Principal instance;
    private Principal() { }

    public static Principal getInstance() {
        if (instance == null) {
            synchronized (Principal.class) {
                if (instance == null) {
                    instance = new Principal();
                }
            }
        }
        return instance;
    }
}

```

Type 4 — Bill Pugh / Static Inner Class (best general choice)

Java guarantees a class loads **once**, **thread-safely**. The inner class isn't loaded until first use — so you get lazy + safe for free, no locks.

```

class Principal {
    private Principal() { }

    // NOT loaded until getInstance() is first called
    private static class Holder {
        private static final Principal INSTANCE = new Principal();
    }

    public static Principal getInstance() {
        return Holder.INSTANCE; // triggers Holder to load - once, safely
    }
}

```

Type 5 — Enum Singleton (safest of all)

Java *guarantees* an enum has exactly one instance, handles threads, and even blocks sneaky attacks (serialization, reflection).

```

enum Principal {
    INSTANCE;
    public void announce(String msg) {
        System.out.println("Principal announces: " + msg);
    }
}
// usage: Principal.INSTANCE.announce("School closes early.");

```

Type	Lazy?	Thread-safe ?	Notes
Eager	No	Yes	Simple, but builds even if unused
Synchronized method	Yes	Yes	Safe but slow (locks every call)
Double-checked locking	Yes	Yes	Fast after first use; needs volatile
Bill Pugh (inner class)	Yes	Yes	BEST general choice — clean & free
Enum	No*	Yes	SAFEST — bulletproof, unusual style

What to say in an interview: “The basic Singleton isn't thread-safe because two threads can both pass the null check and create two objects. The common fixes are double-checked locking with `volatile`, the Bill Pugh static-inner-class approach — which is cleanest — or an enum, which is most robust.” That one sentence shows you understand the *why*, not just the *what*.

2

Factory

Creational — one place that decides what to create

Definition

A class whose only job is to **create objects for you**, so the rest of your code never says `new` directly. You tell the factory *what you want*; it decides which exact class to build and hands it back.

Real-world analogy

A **pizza shop counter**. You walk up and say “one Margherita”. You don't go into the kitchen, get the dough and assemble it yourself. The counter takes your simple request and gives back a finished pizza. **You care about what you want, not how it's made.**

The problem it solves

Without a factory, this ugly block gets copy-pasted all over your codebase. Picture it in **ten files** — add a new dish and you must hunt down and edit all ten.

✗ THE PROBLEM

```
if (type.equals("margherita")) {
    pizza = new Margherita();
} else if (type.equals("farmhouse")) {
    pizza = new Farmhouse();
} else if (type.equals("peppy")) {
    pizza = new PeppyPaneer();
}
// ...repeated in 10 different files. Add a pizza -> edit all 10.
```

Three ingredients: a **common interface** (so all products share a type), the **concrete products**, and the **factory** — the one class that decides which to create.

✓ THE PATTERN • Java

```
// 1. the common interface - every pizza is a "Pizza"
interface Pizza {
    void prepare();
}

// 2. the concrete products
class Margherita implements Pizza {
    public void prepare() {
        System.out.println("Preparing Margherita: cheese + tomato");
    }
}
class Farmhouse implements Pizza {
    public void prepare() {
        System.out.println("Preparing Farmhouse: veggies loaded");
    }
}

// 3. the Factory - the ONE place that knows how to create pizzas
class PizzaFactory {
    public Pizza createPizza(String type) {
        if (type.equalsIgnoreCase("margherita")) return new Margherita();
        if (type.equalsIgnoreCase("farmhouse")) return new Farmhouse();
        throw new IllegalArgumentException("Unknown pizza: " + type);
    }
}

public class Main {
    public static void main(String[] args) {
        PizzaFactory factory = new PizzaFactory();

        // the customer just ASKS - never says `new Margherita()`
        Pizza p1 = factory.createPizza("margherita");
        p1.prepare();

        Pizza p2 = factory.createPizza("farmhouse");
        p2.prepare();
    }
}
```

OUTPUT

```
Preparing Margherita: cheese + tomato
Preparing Farmhouse: veggies loaded
```

Read the main carefully

The customer code **never writes** `new Margherita()`. It says `factory.createPizza("margherita")` and gets back a ready `Pizza`. The type on the left matters too: the customer only knows it has “a `Pizza`”, not which kind — so the calling code stays **loosely coupled** to the general idea, not the specific class. Add a new pizza? Write the class, add one line in the factory — **every other file keeps working, untouched**.

When to use it

When object creation is **complex or repeated**, and you want **one central place** to decide which class gets made — so the rest of your code isn't tangled with new statements and if/else chains.

The SOLID connection

Dependency Inversion — the customer depends on the `Pizza` interface, not concrete classes.

Open/Closed — creation logic sits in one place you extend rather than rewrite everywhere.

One honest caution: Don't wrap a factory around *everything*. If you only create one simple object in one place, plain `new` is clearer. Factories earn their keep when creation is **repeated across the codebase** or **likely to grow**.

3

Builder

Creational — build step-by-step, by name, then build()

Definition

Construct a **complex object step by step**. Instead of cramming everything into one giant constructor, you set each piece one at a time, then say “done — give me the object”.

Real-world analogy

Ordering a **custom Subway sandwich**. You don't shout the whole thing in one breath. You go step by step: pick the bread, add paneer, add cheese, skip the sauce, done. Now imagine ordering it as one fixed sentence with every slot filled in order — you'd lose track of which `true` meant what. Builder fixes exactly that mess.

The problem it solves

Objects with **many fields** — some required, some optional — hit the **telescoping constructor problem**: unreadable call sites, and a constructor for every combination.

✗ THE PROBLEM

```
// Which value is what?? Impossible to read.
User user = new User("Syamala", "Bengaluru", 25, null, true, false, "MCA");

// And you need a constructor for EVERY combination:
User(String name);
User(String name, String city);
User(String name, String city, int age);
User(String name, String city, int age, String phone);
// ...this explodes into dozens of constructors
```

The Builder collects the pieces one at a time. The key trick is **every setter returns this** — the same builder — which is what lets you chain the calls together.

✓ THE PATTERN • Java

```

class User {
    // final = set once, never changed (immutable, safe)
    private final String name;      // required
    private final String city;      // optional
    private final int age;          // optional
    private final String degree;    // optional

    // 1. PRIVATE constructor - only the Builder can create a User
    private User(UserBuilder builder) {
        this.name = builder.name;
        this.city = builder.city;
        this.age = builder.age;
        this.degree = builder.degree;
    }

    public void show() {
        System.out.println(name + " | " + city + " | " + age + " | " + degree);
    }

    // 2. the Builder - collects the pieces
    static class UserBuilder {
        private final String name;    // required -> in constructor
        private String city;          // optional -> via methods
        private int age;
        private String degree;

        public UserBuilder(String name) {
            this.name = name;
        }

        // 3. each method RETURNS THE BUILDER -> enables chaining
        public UserBuilder city(String city) {
            this.city = city;
            return this;
        }
        public UserBuilder age(int age) {
            this.age = age;
            return this;
        }
        public UserBuilder degree(String degree) {
            this.degree = degree;
            return this;
        }

        // 4. the final step - hand back the finished User
        public User build() {
            return new User(this);
        }
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        // build by NAME, in any order
        User u1 = new User.UserBuilder("Syamala")    // required
                .city("Bengaluru")
                .age(25)
                .degree("MCA")
                .build();

        u1.show();

        // skip fields you don't want - NO extra constructor needed
        User u2 = new User.UserBuilder("Sumanth")
                .city("Hyderabad")
                .build();

        u2.show();
    }
}

```

OUTPUT

```

Syamala | Bengaluru | 25 | MCA
Sumanth | Hyderabad | 0 | null

```

The one trick that makes it work: return this

Every value is **labelled** — `city("Bengaluru")`, `age(25)` — so you read it like a sentence instead of counting positions. And `u2` skipped `age` and `degree` entirely, taking defaults (`0`, `null`) — constructor explosion gone.

The chaining works because each method ends with `return this`, handing back the same builder so the next call has something to attach to. Think of it as **passing the same clipboard down a line of people** — each writes their bit and hands the clipboard back. Remove `return this` and the chain snaps: `.city(...)` would return nothing, so `.age(...)` has nothing to call on.

When to use it

When an object has **many fields**, especially a mix of **required and optional** ones, or when you want the object **immutable** (all fields final, set once). Real examples: database configs, HTTP requests, complex DTOs.

The SOLID connection

Single Responsibility — the `User` class holds data, while the *construction logic* lives separately in `UserBuilder`. Two jobs, two classes.

One honest caution: Don't use Builder for a simple object with two or three fields — a plain constructor is clearer. Builder earns its keep when the field count grows or optional combinations multiply. Also note: `final` isn't required for chaining — it's only there to lock the object after building.

4

Adapter

Structural — translate an incompatible interface

Definition

An **Adapter** sits between two things that **can't talk to each other** and translates. It takes an object with the *wrong* interface and makes it look like the interface your code *expects*.

Real-world analogy

You bought a laptop in India (**round two-pin plug**) and you're in the UK (**rectangular three-pin socket**). The plug won't fit, the socket won't change, the laptop won't change. So you buy a **travel adapter** — the middle piece that makes two incompatible shapes work together. It doesn't *do* anything itself; it just **translates one shape into another**.

The problem it solves

Your code expects payments to work one way. Your company buys a third-party library that works another way — different method name, different data type. **You can't change either side**: you can't rewrite your whole codebase, and you don't own the library.

✗ THE PROBLEM

```
// what YOUR code expects:
interface PaymentGateway {
    void pay(double amountInRupees);
}

// what the THIRD-PARTY library gives you:
class ThirdPartyPayment {
    void makePayment(String amountInPaise) { ... }
}

// mismatch 1: method is makePayment, not pay
// mismatch 2: takes a String in PAISE, not a double in RUPEES
```

Three roles: the **Target** (what your code expects), the **Adaptee** (the incompatible class), and the **Adapter** — which implements the target, holds the adaptee inside, and translates between them.

✓ THE PATTERN • Java

```
// 1. TARGET - the interface your code already expects
interface PaymentGateway {
    void pay(double amountInRupees);
}

// 2. ADAPTEE - the third-party class. We CANNOT change this.
class ThirdPartyPayment {
    public void makePayment(String amountInPaise) {
        System.out.println("Third-party paid " + amountInPaise + " paise");
    }
}

// 3. ADAPTER - the translator in the middle
class PaymentAdapter implements PaymentGateway { // looks like what we want

    // it HOLDS the incompatible object inside it
    private ThirdPartyPayment thirdParty;

    public PaymentAdapter(ThirdPartyPayment thirdParty) {
        this.thirdParty = thirdParty;
    }

    @Override
    public void pay(double amountInRupees) {
        // TRANSLATE rupees(double) -> paise(String)
        String amountInPaise = String.valueOf((int)(amountInRupees * 100));
        // then call the third-party method
        thirdParty.makePayment(amountInPaise);
    }
}

public class Main {
    public static void main(String[] args) {
        ThirdPartyPayment thirdParty = new ThirdPartyPayment();

        // wrap the incompatible object inside the adapter
        PaymentGateway gateway = new PaymentAdapter(thirdParty);

        // our code calls pay() the way it always has.
        // it has NO IDEA a third-party library is behind it.
        gateway.pay(500.0);
    }
}
```

OUTPUT

```
Third-party paid 50000 paise
```

The three roles, restated

Look at the type on the left: `PaymentGateway gateway = new PaymentAdapter(...)`. Your code holds the interface it always expected and calls `gateway.pay(500.0)` as before — it doesn't know a differently-shaped class is doing the work. Inside, the adapter converts `500.0` rupees into `"50000"` paise.

Your code speaks “pay in rupees”. **The library** speaks “makePayment in paise”. **The adapter is bilingual** — it implements your language on the outside and speaks theirs on the inside. Two incompatible shapes joined by a translator; neither side changed.

When to use it

When you need an existing class but its **interface doesn't match** what your code needs — and you **can't change either one**. Classic cases: third-party libraries (you don't own them), legacy code (too risky to change), making an old class fit a new interface.

The SOLID connection

Open/Closed — you added support for a new payment library by *writing a new class* (the adapter), not by editing your existing code or the library. Extension, not modification.

One honest caution: Adapter is a **fix for a mismatch you didn't create**. If you control both sides, don't build an adapter — just make the interfaces match in the first place. Adapters wrapping your *own* code usually signal you should have designed the interface better. They earn their place at the boundary with code you can't touch.

5

Decorator

Structural — wrap in layers, each adding behaviour

Definition

Add **new behaviour to an object at runtime** by wrapping it in another object — without changing the original class.

The core idea: **wrapping a gift box inside another gift box**. When asked “what's the total weight?”, each box says “whatever is inside me + my own weight”. The innermost box has nothing inside — that's where the chain stops.

Real-world analogy

Ordering **coffee**. Start with plain espresso (Rs.50). Add milk (+10), caramel (+15), whipped cream (+20). Each addition **wraps** what you had before. Every layer adds its own cost and description, but the espresso underneath was never modified. You can stop after milk, or add caramel twice — **layers stack in any combination**.

The problem it solves

Solving this with inheritance **explodes**. Every new topping *doubles* the number of classes — 3 toppings = 8 classes, 10 toppings = 1024. Decorator gives you 3 toppings = **3 classes**, combined freely at runtime.

✗ THE PROBLEM

```
class Espresso { }
class EspressoWithMilk { }
class EspressoWithMilkAndCaramel { }
class EspressoWithMilkAndCaramelAndCream { }
class EspressoWithCaramelOnly { }
class EspressoWithCreamAndMilk { }
// ...one class for EVERY combination. This is CLASS EXPLOSION.
```

The key trick: a decorator **IS-A Coffee** (so it fits anywhere a Coffee is expected) and **HAS-A Coffee** (so it can wrap one). Take away either and the pattern collapses.

✓ THE PATTERN • Java

```
// 1. COMPONENT - the common interface
interface Coffee {
    String getDescription();
    double getCost();
}

// 2. CONCRETE COMPONENT - the plain base object
class Espresso implements Coffee {
    public String getDescription() { return "Espresso"; }
    public double getCost() { return 50.0; }
}

// 3. DECORATOR - IS-A Coffee (implements) and HAS-A Coffee (holds one)
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee; // the thing being wrapped
    // protected -> children can use it
    public CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }
}

// 4. CONCRETE DECORATORS - each adds its own bit
class Milk extends CoffeeDecorator {
    public Milk(Coffee coffee) { super(coffee); } // parent stores it

    public String getDescription() {
        return coffee.getDescription() + " + Milk"; // ask inner, add mine
    }
    public double getCost() {
        return coffee.getCost() + 10.0; // inner cost + mine
    }
}

class Caramel extends CoffeeDecorator {
    public Caramel(Coffee coffee) { super(coffee); }
    public String getDescription() {
        return coffee.getDescription() + " + Caramel";
    }
    public double getCost() { return coffee.getCost() + 15.0; }
}
}
```

✓ RUNNING IT • Java

```
public class Main {
    public static void main(String[] args) {
        Coffee c1 = new Espresso();
        System.out.println(c1.getDescription() + " = Rs." + c1.getCost());

        // espresso -> milk -> caramel (layers stack)
        Coffee c2 = new Caramel(new Milk(new Espresso()));
        System.out.println(c2.getDescription() + " = Rs." + c2.getCost());

        // double caramel - no new class needed!
        Coffee c3 = new Caramel(new Caramel(new Espresso()));
        System.out.println(c3.getDescription() + " = Rs." + c3.getCost());
    }
}
```

OUTPUT

```
Espresso = Rs.50.0  
Espresso + Milk + Caramel = Rs.75.0  
Espresso + Caramel + Caramel = Rs.80.0
```

What super(coffee) does

Every topping needs to store what it wraps. Rather than writing `this.coffee = coffee;` in all of them, it's written **once** in the parent — and each child calls `super (coffee)` to reuse it. Read `super (coffee)` as: *"I don't want to write the storing code myself — my parent already knows how, so I'll hand the coffee up to them."* Because the parent's field is protected, the child can then use `coffee` directly.

Never re-declare the field in the child — that creates a second, unassigned field that shadows the parent's and causes a `NullPointerException`.

When to use it

When you want to add responsibilities **dynamically** in **flexible combinations**, without a class explosion. Java's own I/O library is literally this: `new BufferedReader (new FileReader ("f.txt"))` — `BufferedReader` wraps a `Reader` and adds buffering. Also: logging, compression, or encryption layers.

The SOLID connection

Open/Closed — add a topping by writing a new class; never touch `Espresso` or existing toppings.

Single Responsibility — each decorator does exactly one thing.

One honest caution: Decorator creates **many small objects and many layers**. Debugging a deeply nested stack (`A(B(C(D(base))))`) is confusing — a stack trace goes through every layer. If you only need one or two fixed combinations, plain inheritance is clearer. Decorator shines when combinations are **many and unpredictable**.

Decorator — the dry run

This is the part worth slowing down on. Take `new Caramel(new Milk(new Espresso()))`. Java builds it **inside-out** — the innermost `new` runs first:

```
Step 1: new Espresso()    -> creates E (plain espresso)
Step 2: new Milk(E)       -> Milk ctor runs, super(E) stores it
                           -> M wraps E
Step 3: new Caramel(M)    -> Caramel ctor runs, super(M) stores it
                           -> C wraps M
```

The structure is an ONION:

```
C (Caramel)
+-- coffee -> M (Milk)
           +-- coffee -> E (Espresso)
               +-- (nothing inside - it's the base)
```

Now trace `c.getCost()`. The call travels **inward** asking, then answers pile up on the way **back out**:

DRY RUN

1. `C.getCost() = coffee.getCost() + 15.0`
-> C's coffee is M, so C must ASK M first. C pauses.
2. `M.getCost() = coffee.getCost() + 10.0`
-> M's coffee is E, so M must ASK E first. M pauses.
3. `E.getCost() = 50.0`
-> E is the BASE. Nothing inside to ask. Returns 50.
-> This is where the chain STOPS.

Now the paused calls resume, from the inside out:

```
E returns 50
M returns 50 + 10 = 60
C returns 60 + 15 = 75  <- final answer
```

Call travels INWARD (asking):

```
C.getCost() --asks--> M.getCost() --asks--> E.getCost()
```

Answers travel OUTWARD (returning):

```
E returns 50 --> M returns 50+10=60 --> C returns 60+15=75
```

The one-sentence summary: each layer, when asked, **asks the layer inside it first, then adds its own bit**. The base layer (Espresso) has nothing inside, so it answers directly — that's where the chain stops and the totals start adding up on the way back out.

Definition

A **single simple front door** to a complicated system. Behind that door there might be many messy classes, but you only ever talk to the one facade — it handles the mess for you.

The word “facade” literally means the *front face* of a building. You see a clean entrance; you don't see the pipes, wiring and machinery behind the wall.

Real-world analogy

You order on **Swiggy** and tap one button: “**Place Order**”. Behind that single tap, a lot happens — payment is processed, the restaurant is notified, a delivery partner is assigned, you get an SMS. But **you** don't call four systems. You tap **one button**. The Swiggy app is the facade.

The problem it solves

Without a facade, *your* code has to know and operate every subsystem itself, in the right order. If any step changes, you fix it everywhere this block appears — and ordering food shouldn't require the caller to understand payments, logistics and messaging.

✗ THE PROBLEM

```
// The client has to do ALL of this, every time, in the correct order:
paymentService.processPayment(userId, amount);
restaurantService.notifyRestaurant(restaurantId, items);
deliveryService.assignDeliveryPartner(userId, restaurantId);
notificationService.sendSms(userId, "Order confirmed!");
```

Three roles: the **subsystem classes** (the complicated machinery), the **facade** (one class coordinating them), and the **client** (your code, which only talks to the facade).

✓ THE PATTERN • Java

```
// 1. SUBSYSTEM CLASSES - the machinery behind Swiggy
class PaymentService {
    public void processPayment(String user, double amount) {
        System.out.println("Payment of Rs." + amount + " done for " + user);
    }
}
class RestaurantService {
    public void notifyRestaurant(String restaurant, String item) {
        System.out.println(restaurant + " notified to prepare: " + item);
    }
}
class DeliveryService {
    public void assignPartner(String user) {
        System.out.println("Delivery partner assigned for " + user);
    }
}
class NotificationService {
    public void sendSms(String user, String message) {
        System.out.println("SMS to " + user + ": " + message);
    }
}

// 2. FACADE - one simple front door over all that machinery
class SwiggyFacade {
    private PaymentService payment;
    private RestaurantService restaurant;
    private DeliveryService delivery;
    private NotificationService notification;

    public SwiggyFacade() {
        this.payment = new PaymentService();
        this.restaurant = new RestaurantService();
        this.delivery = new DeliveryService();
        this.notification = new NotificationService();
    }

    // ONE method that runs all the steps in the right order
    public void placeOrder(String user, String restaurantName,
        String item, double amount) {
        System.out.println("--- Placing your order ---");
        payment.processPayment(user, amount);
        restaurant.notifyRestaurant(restaurantName, item);
        delivery.assignPartner(user);
        notification.sendSms(user, "Your order is confirmed!");
        System.out.println("--- Order placed! ---");
    }
}
}
```

✓ RUNNING IT • Java

```
public class Main {
    public static void main(String[] args) {
        SwiggyFacade swiggy = new SwiggyFacade();

        // The client does ONE thing. All the mess is hidden.
        swiggy.placeOrder("Syamala", "Meghana Foods", "Biryani", 350.0);
    }
}
```

OUTPUT

```
--- Placing your order ---  
Payment of Rs.350.0 done for Syamala  
Meghana Foods notified to prepare: Biryani  
Delivery partner assigned for Syamala  
SMS to Syamala: Your order is confirmed!  
--- Order placed! ---
```

Facade doesn't remove complexity — it hides it

The entire client code is **one line**: `swiggy.placeOrder(...)`. It doesn't know there's a `PaymentService` or a `DeliveryService`, and doesn't know the order of steps.

This is the important bit: the four subsystems are **still there**, still doing their full work. Facade doesn't delete complexity — it puts a **simple face** in front of it. The payoff: if Swiggy adds a loyalty-points step tomorrow, they change it **inside the facade only**, and every caller keeps working untouched. The complexity is real and unavoidable, so the best you can do is **contain it in one place** instead of spreading it across every caller.

When to use it

When you have a **complicated subsystem** and want to give the rest of your code a **simple, clean entry point**. Real examples: a `checkout()` hiding tax, inventory, gateway and receipts; most SDKs you use are facades; a `runPipeline()` hiding ingestion, transformation, validation and load.

The SOLID connection

Single Responsibility — the client's job is “place an order”, not “run a payment gateway”. **Loose coupling** — the client depends only on the facade, so the subsystem can change freely.

One honest caution: A facade should **simplify**, not become a dumping ground. If it grows into a giant class doing everything (a “God object”), you've just moved the mess, not solved it. Keep it focused on being a clean entry point, and let the subsystem classes do the actual work.

7

Strategy

Behavioral — swap the algorithm at runtime

Definition

Define a family of interchangeable “ways to do something” and **swap which one you use while the program is running** — without changing the code that uses it.

Real-world analogy

Google Maps. You want to go from home to office. Maps can compute the route by **car**, **public transport**, **walking**, or **cycling**. The goal is the same — get you there — but the *method of calculating* changes. You tap a different icon and the same feature uses a different strategy. The route-finder doesn't care which mode you picked.

The problem it solves

Without Strategy you get a giant if/else, and every new payment method forces you to reopen and edit that one method.

✗ THE PROBLEM

```
class PaymentService {
    void pay(String type, double amount) {
        if (type.equals("CreditCard")) {
            // credit card logic
        } else if (type.equals("UPI")) {
            // UPI logic
        } else if (type.equals("Wallet")) {
            // wallet logic
        }
        // new method? EDIT this method AGAIN.
    }
}
```

Strategy replaces the if/else with **interchangeable objects** — one per algorithm. Three roles: the **strategy interface**, the **concrete strategies**, and the **context** that uses one and lets you swap it.

✓ THE PATTERN • Java

```
// 1. STRATEGY interface - the common "way to pay"
interface PaymentStrategy {
    void pay(double amount);
}

// 2. CONCRETE STRATEGIES - each is one algorithm
class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using Credit Card");
    }
}
class UpiPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using UPI");
    }
}
class WalletPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using Wallet");
    }
}

// 3. CONTEXT - uses a strategy, and lets you swap it
class ShoppingCart {
    private PaymentStrategy strategy;    // whatever you chose

    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void checkout(double amount) {
        // the cart doesn't know WHICH strategy - it just calls pay()
        strategy.pay(amount);
    }
}
```

✓ RUNNING IT • Java

```
public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        cart.setPaymentStrategy(new UpiPayment());    // pick UPI
        cart.checkout(500.0);

        cart.setPaymentStrategy(new CreditCardPayment()); // swap it
        cart.checkout(1200.0);

        cart.setPaymentStrategy(new WalletPayment());    // swap again
        cart.checkout(300.0);
    }
}
```

OUTPUT

```
Paid Rs.500.0 using UPI
Paid Rs.1200.0 using Credit Card
Paid Rs.300.0 using Wallet
```

Behaviour becomes a plug-in object

The **same cart** paid three different ways. Notice `checkout()` **never changes** — it always just calls `strategy.pay(amount)`. The behaviour changed (UPI → Card → Wallet) but the cart's code stayed exactly the same. You swapped the *algorithm* like swapping a battery — the device didn't change, just what's plugged in.

Adding Net Banking tomorrow? Write `NetBankingPayment` implements `PaymentStrategy` and plug it in. **You never touch `ShoppingCart` or any existing payment class.** That's the win.

When to use it

When you have **several ways to do the same task** and want to choose or switch between them at runtime. Real examples: payment methods, sorting algorithms (by price/rating/distance), compression (zip/rar/gzip), route-finding.

The SOLID connection

This one is direct. **Open/Closed** — add an algorithm by writing a new class, never editing existing ones.

Dependency Inversion — the cart depends on the `PaymentStrategy` interface, not concrete classes. Strategy is *literally* the Open/Closed payment example, now with a name.

One honest caution: Strategy adds a class per algorithm. If you have only **two fixed options that will never grow** (say on/off), a simple if/else or boolean is clearer — don't create an interface and three classes for something trivial. Strategy earns its place when options are **many, likely to grow**, or need **runtime swapping**.

Definition

Let one object **notify many others automatically** when something changes. The watched object doesn't know who's listening — it just announces, and everyone subscribed gets told.

Real-world analogy

A **YouTube channel**. You subscribe. When the creator uploads a new video, **you get a notification** — automatically. So does every other subscriber. The creator doesn't call each of you personally; they just upload, and the system tells everyone subscribed. People can subscribe or unsubscribe anytime, and the creator's job doesn't change.

The problem it solves

Without Observer, the channel has to know every subscriber by name and call them one by one — hard-wired, needs editing for every new subscriber, and can't change while running.

✗ THE PROBLEM

```
class YouTubeChannel {
    void uploadVideo(String title) {
        syamala.notify(title);
        sumanth.notify(title);
        anjali.notify(title);
        // new subscriber?      EDIT this method again.
        // someone unsubscribes? EDIT this method again.
    }
}
```

Observer fixes all of it: the channel keeps a **list** and just loops over it. Three roles: the **observer interface**, the **subject** (keeps the list), and the **concrete observers**.

✓ THE PATTERN • Java

```

import java.util.ArrayList;
import java.util.List;

// 1. OBSERVER interface - anyone who wants to be notified
interface Subscriber {
    void update(String videoTitle);
}

// 2. SUBJECT - the thing being watched
class YouTubeChannel {
    private String channelName;

    // the LIST - this replaces hard-coded names
    private List<Subscriber> subscribers = new ArrayList<>();

    public YouTubeChannel(String channelName) {
        this.channelName = channelName;
    }

    public void subscribe(Subscriber s) { subscribers.add(s); }
    public void unsubscribe(Subscriber s) { subscribers.remove(s); }

    // tell EVERYONE in the list
    private void notifySubscribers(String videoTitle) {
        for (Subscriber s : subscribers) {
            s.update(videoTitle); // doesn't know WHO - just calls update()
        }
    }

    // the trigger - when this happens, everyone gets told
    public void uploadVideo(String videoTitle) {
        System.out.println "[" + channelName + " uploaded: " + videoTitle + " ]";
        notifySubscribers(videoTitle);
    }
}

// 3. CONCRETE OBSERVER - an actual subscriber
class User implements Subscriber {
    private String name;
    public User(String name) { this.name = name; }

    public void update(String videoTitle) {
        System.out.println(" " + name + " notified: watch \"" + videoTitle + "\"");
    }
}

```

✓ RUNNING IT • Java

```

public class Main {
    public static void main(String[] args) {
        YouTubeChannel channel = new YouTubeChannel("CodeCentral");

        Subscriber syamala = new User("Syamala");
        Subscriber sumanth = new User("Sumanth");
        Subscriber anjali = new User("Anjali");

        channel.subscribe(syamala);          // subscribe at runtime
        channel.subscribe(sumanth);
        channel.subscribe(anjali);

        channel.uploadVideo("Observer Pattern Explained");

        channel.unsubscribe(anjali);        // unsubscribe at runtime

        channel.uploadVideo("Strategy Pattern Explained");
    }
}

```

OUTPUT

```

[CodeCentral uploaded: Observer Pattern Explained]
  Syamala notified: watch "Observer Pattern Explained"
  Sumanth notified: watch "Observer Pattern Explained"
  Anjali notified: watch "Observer Pattern Explained"

[CodeCentral uploaded: Strategy Pattern Explained]
  Syamala notified: watch "Strategy Pattern Explained"
  Sumanth notified: watch "Strategy Pattern Explained"

```

The channel doesn't know who's listening

Look at the **second upload** — Anjali is **gone**, because she unsubscribed. But `uploadVideo()` **never changed**. The behaviour changed because the **list** changed, not the code.

The heart of it is for `(Subscriber s : subscribers) s.update(title);` — the channel holds the **interface**, so it has no idea whether each listener is a `User`, a mobile app, an email service or a logger. It just says “update yourself”. That's why you can add `class EmailAlert implements Subscriber` tomorrow and `channel.subscribe(new EmailAlert())` works immediately — **without touching YouTubeChannel**.

When to use it

When **one change needs to inform many things**, and you don't want the source hard-wired to its listeners. Real examples: button clicks in UI, pub/sub messaging (Kafka, RabbitMQ), Spring's `ApplicationEvent`, stock price updates, order-status notifications.

The SOLID connection

Open/Closed — add a new subscriber type by writing a new class; never edit the channel.

Dependency Inversion — the channel depends on the `Subscriber` interface, not concrete classes.

One honest caution: Two real gotchas. **(1) The lapsed listener problem** — if a subscriber is destroyed but you forget to `unsubscribe()`, the list still holds a reference, so it never gets garbage collected (a memory leak). **(2) Notification order isn't guaranteed** — if A must run before B, `Observer` is the wrong tool; you're describing a sequence, not a broadcast. Also keep chains shallow — A notifies B notifies C gets very hard to debug.

All eight at a glance

	Pattern	Family	One-line memory hook
1	Singleton	Creational	one shared instance, locked new
2	Factory	Creational	one place that decides what to create
3	Builder	Creational	build step-by-step, by name, then build()
4	Adapter	Structural	translate an incompatible interface
5	Decorator	Structural	wrap in layers, each adding behaviour
6	Facade	Structural	one simple door over a messy subsystem
7	Strategy	Behavioral	swap the algorithm at runtime
8	Observer	Behavioral	broadcast to many listeners

The three “wrapper” patterns — don't mix them up

Adapter, Decorator and Facade all wrap an object, so they blur together. Here is the clean separation:

Pattern	What it wraps	Why
Adapter	one incompatible class	to CHANGE its interface so it fits
Decorator	one object (stackable)	to ADD behaviour, keeping the same interface
Facade	a whole subsystem of many classes	to SIMPLIFY — one easy entry point

Quick way to remember: Adapter **translates**. Decorator **enhances**. Facade **simplifies**.

Strategy vs Observer — the behavioural pair

	Strategy	Observer
Relationship	one context, ONE plugged-in algorithm	one subject, MANY listeners
Question it answers	“how should this task be done?”	“who needs to know this happened?”
Direction	you push a strategy IN	the subject pushes news OUT
Example	pay by UPI or card	new video -> tell all subscribers

Quick way to remember: Strategy **swaps behaviour**. Observer **broadcasts events**.

The one thread

*Every pattern is a different tactic for the **same goal** as SOLID: reduce how far a single change ripples through your code. Notice how often the answer was the same move — program to an interface, so you add a class instead of editing one. That's Open/Closed and Dependency Inversion appearing over and over. The patterns really are SOLID made concrete.*